



 **SIMATS**
ENGINEERING  **SIMATS**
Department of Information Systems and Management

INTRODUCTION

- The Substitution Method is used to solve and prove recurrence relations.
- It assumes a solution and verifies it using mathematical induction.
- It is widely used in the Design and Analysis of Algorithms (DAA).
- It helps determine the time complexity of recursive algorithms.
- It is commonly applied to divide-and-conquer algorithms.
- For logarithmic growth, the recurrence relation is $T(n) = T(n/2) + 1$.
- Since the problem size is halved in each step, the time complexity is $O(\log n)$.



 GPS Map Camera

**Mevalurkuppam, Tamil Nadu,
India** 

22h8+5h2, Mevalurkuppam, Tamil Nadu 602105,
India

Lat 13.027487° Long 80.016687°

Wednesday, 22/07/2026 10:30 AM GMT +05:30



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Substitution Method for Logarithmic Growth

GUIDED BY

Dr.K.RAJAGOPAL
SIMATS Engineering

STUDENT

SRIMATHI.H(192524403)
Dept of AIDS
SIMATS Engineering



SIMATS
ENGINEERING



SIMATS
Savitribha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

INTRODUCTION

- The Substitution Method is used to solve and prove recurrence relations.
- It assumes a solution and verifies it using mathematical induction.
- It is widely used in the Design and Analysis of Algorithms (DAA).
- It helps determine the time complexity of recursive algorithms.
- It is commonly applied to divide-and-conquer algorithms.
- For logarithmic growth, the recurrence relation is $T(n) = T(n/2) + 1$.
- Since the problem size is halved in each step, the time complexity is $O(\log n)$.

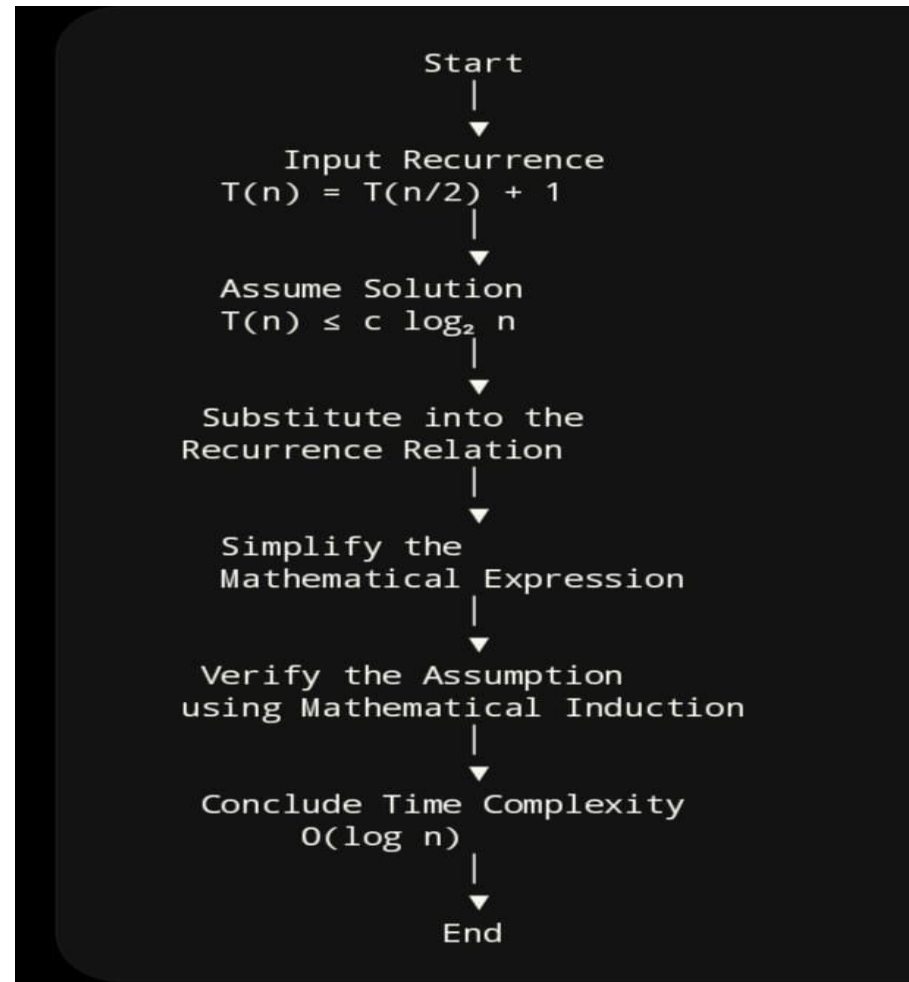
PROBLEM STATEMENT

- Analyze the recurrence relation $T(n) = T(n/2) + 1$.
- Apply the Substitution Method to solve the recurrence.
- Assume a solution and verify it using mathematical induction.
- Simplify the recurrence relation step by step.
- Prove that the assumed solution satisfies the recurrence.
- Determine the time complexity as $O(\log n)$.

ALGORITHM

- Start with the recurrence relation $T(n) = T(n/2) + 1$.
- Assume the solution $T(n) \leq c \log_2 n$, where c is a positive constant.
- Substitute the assumed solution into the recurrence relation.
- Simplify the expression using logarithmic properties.
- Verify that the inequality holds for a suitable value of c .
- Conclude that the recurrence relation has a time complexity of $O(\log n)$.

ARCHITECTURE DIAGRAM



IMPLEMENTATION

```
main.py Output [Icons] [Run]
1 def recurrence(n):
2     # Base case
3     if n <= 1:
4         return 0
5
6     # Recursive case
7     return recurrence(n // 2) + 1
8
9
10 # Driver code
11 n = int(input("Enter the value of n: "))
12 result = recurrence(n)
13
14 print("Number of recursive calls:",
15       result)
16 print("Time Complexity: O(log n)")
```

```
main.py Output [Icons] [Run]
Enter the value of n: 16
Number of recursive calls: 4
Time Complexity: O(log n)

=== Code Execution Successful ===
```

COMPLEXITY ANALYSIS

Parameter	Analysis
Recurrence Relation	$T(n) = T(n/2) + 1$
Time Complexity	$O(\log n)$
Space Complexity	$O(\log n)$ (Recursive Call Stack)
Best Case	$O(\log n)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$
Reason	The input size is reduced by half in each recursive call, therefore the number of recursive calls is proportional to $\log_2 n$.

CONCLUSION

- The Substitution Method is an effective technique for proving the time complexity of recurrence relations.
- For the recurrence $T(n) = T(n/2) + 1$, the problem size is reduced by half at each recursive step.
- The recurrence is proven to have $O(\log n)$ time complexity using mathematical induction.
- This method is widely used to analyze divide-and-conquer algorithms such as Binary Search.



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

REFERENCE

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, 4th Edition, MIT Press, 2022.
- Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, Fundamentals of Computer Algorithms, 2nd Edition, Universities Press.
- Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, The Design and Analysis of Computer Algorithms, Addison-Wesley.
- Donald E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, Addison-Wesley.

THANK YOU